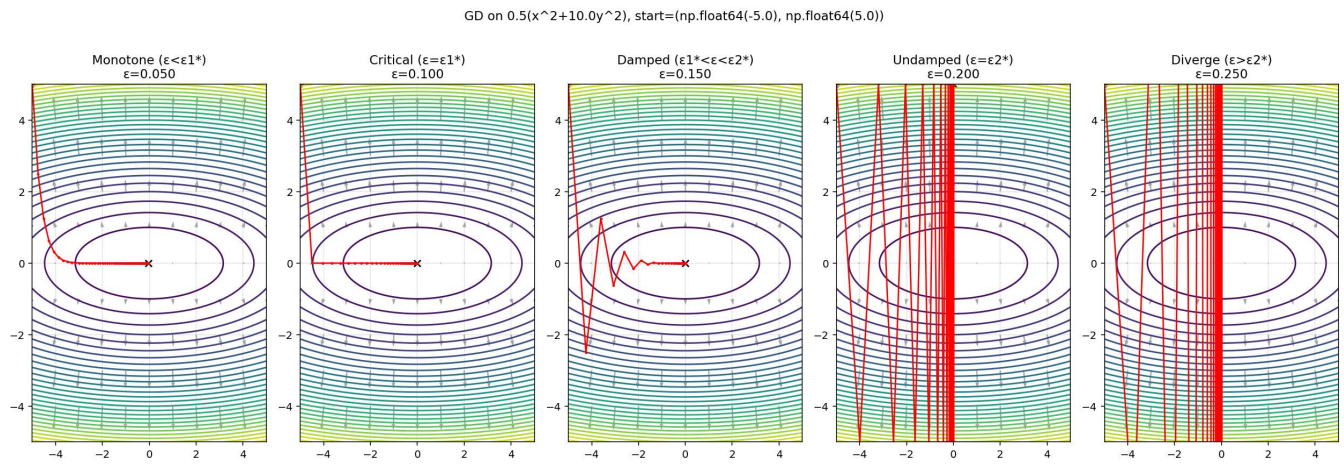
```
import numpy as np
import matplotlib.pyplot as plt

# Quadratic with anisotropy gamma
def f_vec(z, gamma=10.0):
    x, y = z[..., 0], z[..., 1]
    return 0.5*(x**2 + gamma*y**2)

def grad_f_vec(z, gamma=10.0):
    x, y = z[..., 0], z[..., 1]
    return np.stack([x, gamma*y], axis=-1)

# Plot contour for a 2D scalar function f_vec
def plot_2d_contour(f_vec, xlim, ylim, gridsize=100, gamma=10.0):
    xx = np.linspace(xlim[0], xlim[1], gridsize)
    yy = np.linspace(ylim[0], ylim[1], gridsize)
    X, Y = np.meshgrid(xx, yy)
    Z = np.stack([X, Y], axis=-1)
    Zf = f_vec(Z, gamma=gamma)
    c = plt.contour(X, Y, Zf, levels=30)
    return c

# Plot gradient field
def plot_2d_gradient_field(f_vec, xlim, ylim, gridsize=11, gamma=10.0):
    xx = np.linspace(xlim[0], xlim[1], gridsize)
    yy = np.linspace(ylim[0], ylim[1], gridsize)
    X, Y = np.meshgrid(xx, yy)
    Z = np.stack([X, Y], axis=-1)
    GZ = grad_f_vec(Z, gamma=gamma).reshape(-1,2)
    maxnorm = np.maximum(1e-8, np.sqrt((GZ**2).sum(axis=1)).max())
    U = GZ[:,0].reshape((gridsize,gridsize))/maxnorm
    V = GZ[:,1].reshape((gridsize,gridsize))/maxnorm
    plt.quiver(X, Y, U, V, units='xy', scale=1, color='gray', alpha=0.7)

# Optimizers
def run_gd_step(z, eps, gamma=10.0):
    return z - eps * grad_f_vec(z, gamma=gamma)

def run_adam(z0, alpha, T, gamma=10.0, b1=0.9, b2=0.999, eps=1e-8):
    z = np.array(z0, dtype=float)
    m = np.zeros(2); v = np.zeros(2)
    traj = [z.copy()]
    for t in range(1, T+1):
        g = grad_f_vec(z, gamma=gamma)
        m = b1*m + (1-b1)*g
        v = b2*v + (1-b2)*(g**2)
        mhat = m/(1-b1**t)
        vhat = v/(1-b2**t)
        z = z - alpha * mhat/(np.sqrt(vhat)+eps)
        traj.append(z.copy())
    return np.array(traj)
```

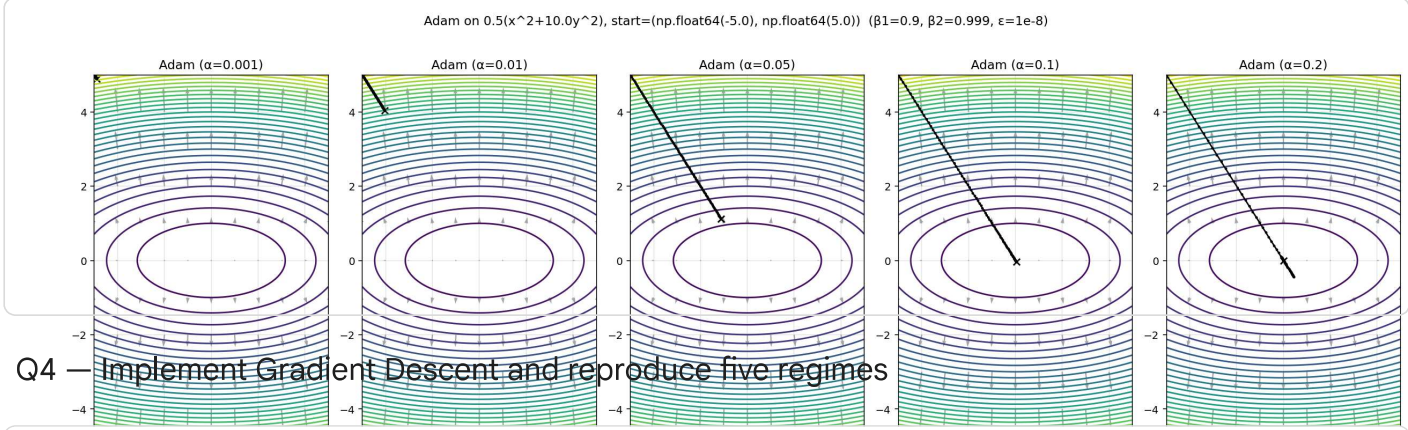
Reproduce Fig.1

```
gamma = 10.0
start = np.array([-5.0, 5.0])
xlim = [-5, 5]; ylim = [-5, 5]
T = 100
eps_list = [0.05, 0.10, 0.15, 0.20, 0.25]
titles = ["Monotone (ε<ε1*)","Critical (ε=ε1*)","Damped (ε1*<ε<ε2*)","Undamped (ε=ε2*)","Diverge (ε>ε2*)"]

fig = plt.figure(figsize=(18,6), dpi=140)
for i, (eps, ttl) in enumerate(zip(eps_list, titles), start=1):
    z = start.copy()
    traj = [z.copy()]
    for _ in range(T):
        z = run_gd_step(z, eps, gamma=gamma)
        traj.append(z.copy())
    traj = np.array(traj)

    ax = plt.subplot(1,5,i)
    plot_2d_contour(f_vec, xlim, ylim, gridsize=150, gamma=gamma)
```

```
    plot_2d_gradient_field(f_vec, xlim, ylim, gridsize=11, gamma=gamma)
    ax.plot(traj[:,0], traj[:,1], 'r.-', lw=1.4, ms=3)
    ax.scatter([traj[0,0]],[traj[0,1]], c='k', marker='o', s=25)
    ax.scatter([traj[-1,0]],[traj[-1,1]], c='k', marker='x', s=40)
    ax.set_xlim(xlim); ax.set_ylim(ylim); ax.grid(alpha=.3)
    ax.set_title(f"{ttl}\nε={eps:.3f}")
plt.suptitle(f"GD on 0.5(x^2+{gamma}y^2), start={tuple(start)}", y=1.02)
plt.tight_layout()
```

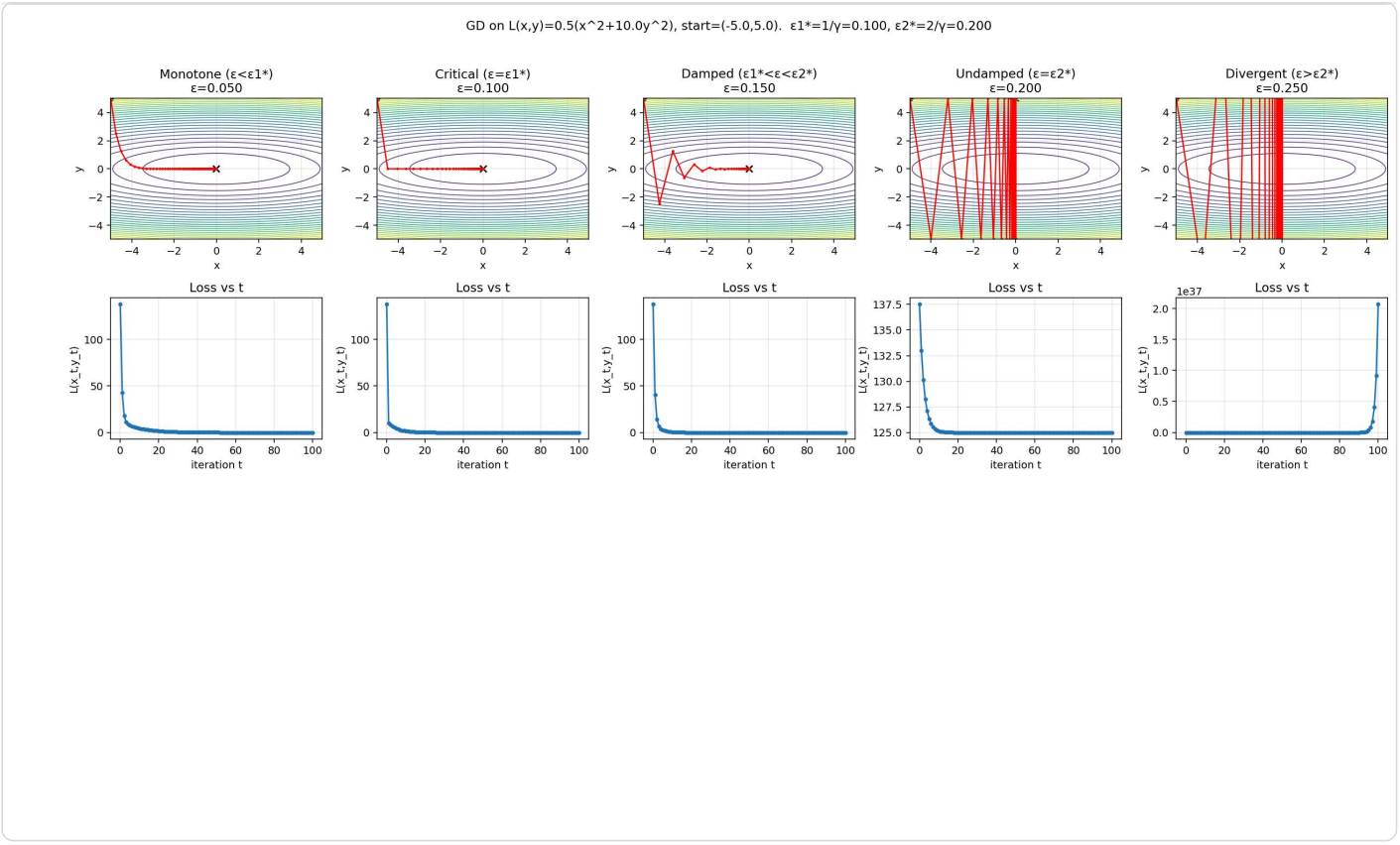



```
alphas = [0.001, 0.01, 0.05, 0.1, 0.2]
fig = plt.figure(figsize=(18,6), dpi=140)
for i, a in enumerate(alphas, start=1):
    traj = run_adam(start, a, T, gamma=gamma, b1=0.9, b2=0.999, eps=1e-8)
    ax = plt.subplot(1,5,i)
    plot_2d_contour(f_vec, xlim, ylim, gridsize=150, gamma=gamma)
    plot_2d_gradient_field(f_vec, xlim, ylim, gridsize=11, gamma=gamma)
    ax.plot(traj[:,0], traj[:,1], 'k.-', lw=1.4, ms=3)
    ax.scatter([traj[0,0]],[traj[0,1]], c='k', marker='o', s=25)
    ax.scatter([traj[-1,0]],[traj[-1,1]], c='k', marker='x', s=40)
    ax.set_xlim(xlim); ax.set_ylim(ylim); ax.grid(alpha=.3)
    ax.set_title(f"Adam (α={a})")
plt.suptitle(f"Adam on 0.5(x^2+{gamma}y^2), start={tuple(start)}  (β1=0.9, β2=0.999, ε=1e-8)", y=1.02)
plt.tight_layout()
```



Q4 — Implement Gradient Descent and reproduce five regimes

```
import numpy as np
import matplotlib.pyplot as plt

γ = 10.0
x0, y0 = -5.0, 5.0
T = 100

def L(x, y, gamma=γ):
    return 0.5*(x**2 + gamma*(y**2))

def grad_L(x, y, gamma=γ):
    return np.array([x, gamma*y], dtype=float)

def run_gd(x0, y0, eps, T, gamma=γ):
    x, y = x0, y0
    traj = [(x, y)]
    losses = [L(x, y, gamma)]
    for _ in range(T):
        g = grad_L(x, y, gamma)
        x -= eps * g[0]
        y -= eps * g[1]
        traj.append((x, y))
        losses.append(L(x, y, gamma))
    return np.array(traj), np.array(losses)

xs = np.linspace(-5, 5, 300)
ys = np.linspace(-5, 5, 300)
XX, YY = np.meshgrid(xs, ys)
ZZ = 0.5*(XX**2 + γ*(YY**2))

ε1, ε2 = 1/γ, 2/γ
eps_list = [0.05, ε1, 0.15, ε2, 0.25]
titles   = ["Monotone (ε<ε1*)", "Critical (ε=ε1*)", "Damped (ε1*<ε<ε2*)",
            "Undamped (ε=ε2*)", "Divergent (ε>ε2*)"]

fig = plt.figure(figsize=(18, 6), dpi=140)
for i, (eps, ttl) in enumerate(zip(eps_list, titles), start=1):
    traj, losses = run_gd(x0, y0, eps, T)
    ax1 = plt.subplot(2, 5, i)
    ax1.contour(XX, YY, ZZ, levels=25, linewidths=0.7)
    ax1.plot(traj[:,0], traj[:,1], 'r.-', lw=1.3, ms=3)
    ax1.scatter([traj[0,0]],[traj[0,1]], c='k', marker='o', s=25)
    ax1.scatter([traj[-1,0]],[traj[-1,1]], c='k', marker='x', s=40)
    ax1.set_xlim(-5,5); ax1.set_ylim(-5,5); ax1.grid(alpha=0.3)
    ax1.set_title(f"{ttl}\nε={eps:.3f}")
    ax1.set_xlabel("x"); ax1.set_ylabel("y")

    ax2 = plt.subplot(2, 5, 5+i)
    ax2.plot(np.arange(T+1), losses, '.-')
    ax2.set_xlabel("iteration t"); ax2.set_ylabel("L(x_t,y_t)")
    ax2.set_title("Loss vs t"); ax2.grid(alpha=0.3)

fig.suptitle(f"GD on L(x,y)=0.5(x^2+{γ}y^2), start=({x0},{y0}).  ε1*=1/γ={ε1:.3f}, ε2*=2/γ={ε2:.3f}", y=1.03)
fig.tight_layout()
```



Q6 — Implement Adam and vary learning rate α; compare behavior to GD

```
import numpy as np
import matplotlib.pyplot as plt

γ = 10.0
x0, y0 = -5.0, 5.0
T = 100

def L(x, y, gamma=γ):
    return 0.5*(x**2 + gamma*(y**2))

def grad_L(x, y, gamma=γ):
    return np.array([x, gamma*y], dtype=float)

def run_adam(x0, y0, α, T, gamma=γ, β1=0.9, β2=0.999, ε=1e-8):
    x, y = x0, y0
    m = np.zeros(2); v = np.zeros(2)
    traj = [(x, y)]
    losses = [L(x, y, gamma)]
    for t in range(1, T+1):
        g = grad_L(x, y, gamma)
        m = β1*m + (1-β1)*g
        v = β2*v + (1-β2)*(g**2)
        m_hat = m / (1 - β1**t)
        v_hat = v / (1 - β2**t)
        step = α * m_hat / (np.sqrt(v_hat) + ε)
        x -= step[0]; y -= step[1]
        traj.append((x, y))
        losses.append(L(x, y, gamma))
    return np.array(traj), np.array(losses)

xs = np.linspace(-5, 5, 300)
ys = np.linspace(-5, 5, 300)
XX, YY = np.meshgrid(xs, ys)
ZZ = 0.5*(XX**2 + γ*(YY**2))

alphas = [0.001, 0.01, 0.05, 0.10, 0.20]
fig = plt.figure(figsize=(18, 6), dpi=140)

for i, α in enumerate(alphas, start=1):
```

```
    traj, losses = run_adam(x0, y0, α, T)
    ax1 = plt.subplot(2, 5, i)
    ax1.contour(XX, YY, ZZ, levels=25, linewidths=0.7)
    ax1.plot(traj[:,0], traj[:,1], 'k.-', lw=1.3, ms=3)
    ax1.scatter([traj[0,0]],[traj[0,1]], c='k', marker='o', s=25)
    ax1.scatter([traj[-1,0]],[traj[-1,1]], c='k', marker='x', s=40)
    ax1.set_xlim(-5,5); ax1.set_ylim(-5,5); ax1.grid(alpha=0.3)
    ax1.set_title(f"Adam traj (α={α})")
    ax1.set_xlabel("x"); ax1.set_ylabel("y")

    ax2 = plt.subplot(2, 5, 5+i)
    ax2.plot(np.arange(T+1), losses, '.-')
    ax2.set_xlabel("iteration t"); ax2.set_ylabel("L(x_t,y_t)")
    ax2.set_title("Loss vs t"); ax2.grid(alpha=0.3)

fig.suptitle(f"Adam on L(x,y)=0.5(x^2+{γ}y^2), start=({x0},{y0})  (β1=0.9, β2=0.999, ε=1e-8)", y=1.03)
fig.tight_layout()
```